

THEORY ASSIGNMENT

Course: Full Stack Web Development
Course Code: 23CM4121
Topic: Node.js Modules (os, path, fs)
Student Name: P.Raghu Ram
Roll No.: A24126552102
Date: 30-08-2026

1. QUESTION

What are Node.js modules? Explain the purpose of the *os*, *path*, and *fs* modules with suitable examples.

2. ANSWER

1. What are Node.js Modules?

In Node.js, modules act as independent, reusable blocks of code that encapsulate specific functionality. By breaking an application down into modules, developers can write cleaner, more maintainable code without polluting the global namespace. Node.js uses the CommonJS module system (using *require()* and *module.exports*), though ES modules (using *import* and *export*) are also supported.

Modules can be categorized into three main types:

- **Core/Built-in Modules:** Included with Node.js installation natively (e.g., *os*, *fs*, *path*, *http*).
- **Local Modules:** Custom modules created locally within your application.
- **Third-party Modules:** Downloaded via Node Package Manager (NPM).

2. The 'os' Module

Purpose: The *os* module provides operating system-related utility methods and properties. It allows developers to interact with the underlying OS to fetch details like architecture, memory, CPU info, and network interfaces.

```
const os = require('os');

console.log("OS Platform:", os.platform()); // e.g., 'win32' or 'linux'
console.log("Total Memory:", os.totalmem() / (1024 * 1024 * 1024), "GB");
console.log("Free Memory:", os.freemem() / (1024 * 1024 * 1024), "GB");
console.log("System Uptime:", os.uptime(), "seconds");
```

3. The 'path' Module

Purpose: The *path* module provides utilities for working with file and directory paths. It is highly essential because different operating systems use different path separators (e.g., Windows uses `\` while Linux/macOS uses `/`). The module standardizes these operations securely.

```
const path = require('path');

const filePath = '/users/test/docs/file.txt';

console.log("Base Name:", path.basename(filePath)); // file.txt
console.log("Directory Name:", path.dirname(filePath)); // /users/test/docs
console.log("Extension:", path.extname(filePath)); // .txt

// Cross-platform safe path joining
```

```
const safePath = path.join(__dirname, 'views', 'index.html');  
console.log("Joined Path:", safePath);
```

4. The 'fs' Module (File System)

Purpose: The `fs` module enables interaction with the file system on a computer. It provides APIs modeled on standard POSIX functions allowing developers to read, write, append, update, and delete files synchronously or asynchronously.

```
const fs = require('fs');

// 1. Writing to a file (Asynchronous)
fs.writeFile('notes.txt', 'Learning Node.js modules!', (err) => {
  if (err) throw err;
  console.log('File successfully written!');

  // 2. Reading from the file (Asynchronous)
  fs.readFile('notes.txt', 'utf8', (err, data) => {
    if (err) throw err;
    console.log('File Contents:', data);
  });
});

// Using Synchronous methods (Blocks execution)
try {
  fs.writeFileSync('sync-file.txt', 'Synchronous Write');
  const content = fs.readFileSync('sync-file.txt', 'utf8');
  console.log("Sync Read:", content);
} catch (err) {
  console.error("Error:", err);
}
```